

How Rate Limiting Works?

The number of queries a user or system can make to a service in a predetermined period of time can be managed by rate limitation. A service might permit 100 requests per minute, for instance. Any additional requests will be blocked or slowed down by the system until the time window is reset once that limit is reached.

- This helps prevent things like abuse, bot attacks, or overloading the server, while also ensuring that all users get a fair chance to access the service.
- It's often done using different methods like token bucket or sliding window, but the goal is always the same: to keep the system running smoothly and protect it from excessive traffic.

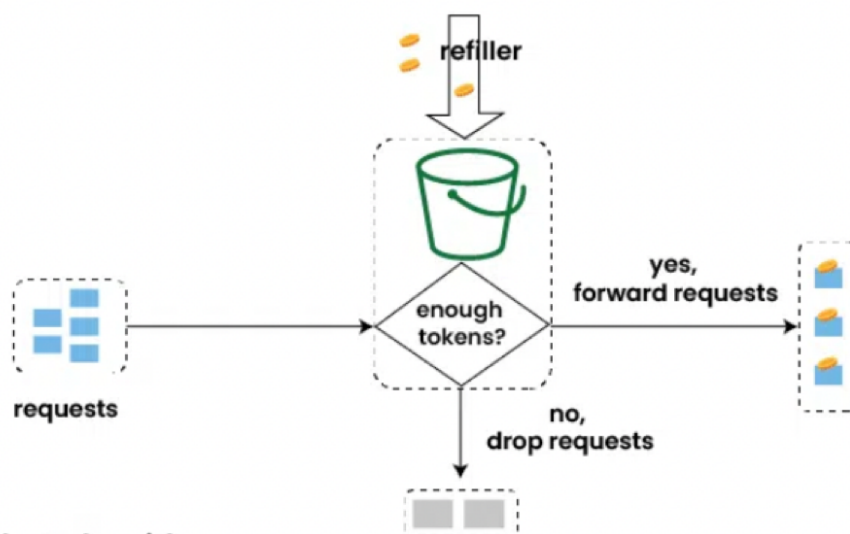
Let's understand the rate limiting algorithms to know how does it works using different algorithms:

Rate Limiting Algorithms

Several rate limiting algorithms are commonly used in system design to control the rate of incoming requests or actions. Below are some popular rate limiting algorithms:

1. Token Bucket Algorithm

- The token bucket algorithm allocates tokens at a fixed rate into a "bucket."
- Each request consumes a token from the bucket, and requests are only allowed if there are sufficient tokens available.
- Unused tokens are stored in the bucket, up to a maximum capacity.
- This algorithm provides a simple and flexible way to control the rate of requests and smooth out bursts of traffic



Token Bucket Algorithm

```
import time

class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity # max number of tokens
        self.tokens = capacity # current number of tokens
        self.refill_rate = refill_rate # tokens added per second
        self.last_refill_time = time.time()

    def refill(self):
        current_time = time.time()
        elapsed_time = current_time - self.last_refill_time
```

```

# Refill the bucket based on elapsed time
new_tokens = elapsed_time * self.refill_rate
self.tokens = min(self.capacity, self.tokens + new_tokens)
self.last_refill_time = current_time

def allow_request(self):
    self.refill()
    if self.tokens >= 1:
        self.tokens -= 1
        return True
    return False

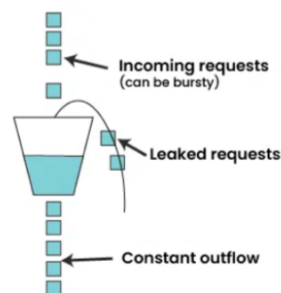
# Example usage:
bucket = TokenBucket(5, 1) # 5 tokens max, refill 1 token per second

for _ in range(10):
    if bucket.allow_request():
        print("Request allowed")
    else:
        print("Request denied")
    time.sleep(0.5) # Simulate request interval

```

2. Leaky Bucket Algorithm

- The leaky bucket algorithm models a bucket with a leaky hole, where requests are added at a constant rate and leak out at a controlled rate.
- Incoming requests are added to the bucket, and if the bucket exceeds a certain capacity, excess requests are either delayed or rejected.
- This algorithm provides a way to enforce a maximum request rate while allowing some burstiness in traffic.



Leaky Bucket Algorithm

```

import time

class LeakyBucket:
    def __init__(self, capacity, leak_rate):
        self.capacity = capacity # max bucket size
        self.water = 0 # current amount of "water" in the bucket
        self.leak_rate = leak_rate # rate at which water leaks per second
        self.last_time = time.time()

    def leak(self):
        current_time = time.time()
        elapsed_time = current_time - self.last_time
        self.water = max(0, self.water - elapsed_time * self.leak_rate)

```

```

self.last_time = current_time

def allow_request(self):
    self.leak()
    if self.water < self.capacity:
        self.water += 1 # Add 1 unit of "water" for each request
        return True
    return False

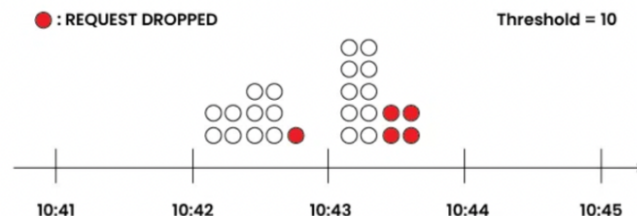
# Example usage:
bucket = LeakyBucket(5, 1) # max 5 requests, leak 1 per second

for _ in range(10):
    if bucket.allow_request():
        print("Request allowed")
    else:
        print("Request denied")
    time.sleep(0.5) # Simulate request interval

```

3. Fixed Window Counting Algorithm

- The fixed window counting algorithm tracks the number of requests within a fixed time window (e.g., one minute, one hour).
- Requests exceeding a predefined threshold within the window are rejected or delayed until the window resets.
- This algorithm provides a straightforward way to limit the rate of requests over short periods, but it may not handle bursts of traffic well.



Fixed Window Counting Algorithm

```

import time

class FixedWindowCounter:
    def __init__(self, limit, window_size):
        self.limit = limit # max requests allowed
        self.window_size = window_size # time window in seconds
        self.count = 0 # current count of requests in the window
        self.start_time = time.time()

    def allow_request(self):
        current_time = time.time()
        if current_time - self.start_time > self.window_size:
            # Reset the window
            self.start_time = current_time
            self.count = 0
        if self.count < self.limit:
            self.count += 1
            return True

```



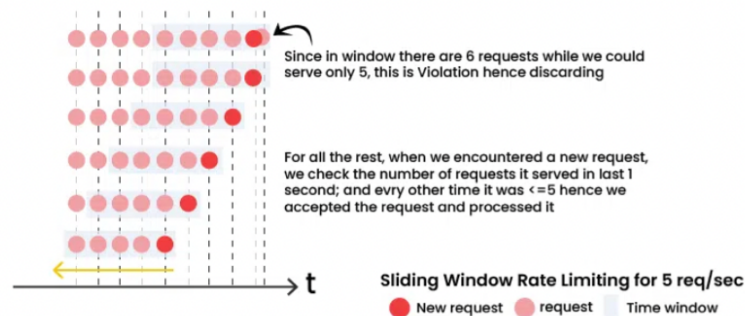
```
return False
```

```
# Example usage:
window = FixedWindowCounter(5, 10) # 5 requests per 10 seconds

for _ in range(10):
    if window.allow_request():
        print("Request allowed")
    else:
        print("Request denied")
    time.sleep(2) # Simulate request interval
```

4. Sliding Window Log Algorithm

- The sliding window log algorithm maintains a log of timestamps for each request received.
- Requests older than a predefined time interval are removed from the log, and new requests are added.
- The rate of requests is calculated based on the number of requests within the sliding window.
- This algorithm allows for more precise rate limiting and better handling of bursts of traffic compared to fixed window counting.



Sliding Window Log Algorithm

```
import time
from collections import deque

class SlidingWindowLog:
    def __init__(self, limit, window_size):
        self.limit = limit # max requests allowed
        self.window_size = window_size # time window in seconds
        self.requests = deque() # stores timestamps of requests

    def allow_request(self):
        current_time = time.time()

        # Remove requests outside the window
        while self.requests and self.requests[0] <= current_time - self.window_size:
            self.requests.popleft()

        if len(self.requests) < self.limit:
            self.requests.append(current_time)
            return True
        return False
```

```
# Example usage:
sliding_window = SlidingWindowLog(5, 10) # 5 requests per 10 seconds

for _ in range(10):
    if sliding_window.allow_request():
        print("Request allowed")
```

```

        print("Request allowed")
    else:
        print("Request denied")
        time.sleep(2) # Simulate request interval

```

Client-Side vs. Server-Side Rate Limiting

Below are the differences between Client-Side and Server-Side Rate Limiting:

Aspect	Client-Side Rate Limiting	Server-Side Rate Limiting
Location of Enforcement	Enforced by the client application or client library.	Enforced by the server infrastructure or API gateway.
Request Control	Requests are throttled or delayed before reaching the server.	Requests are processed by the server, which decides whether to accept, reject, or delay them based on predefined rules.
Flexibility	Limited flexibility as it relies on client-side implementation and configuration.	Offers greater flexibility as rate limiting rules can be centrally managed and adjusted on the server side without client-side changes.
Security	Less secure as it can be bypassed or manipulated by clients.	More secure as enforcement is centralized and controlled by the server, reducing the risk of abuse or exploitation.
Scalability	May impact client performance and scalability, especially in distributed environments with a large number of clients.	Better scalability as rate limiting can be applied globally across all clients and adjusted dynamically based on server load and resource availability.

Rate Limiting in Different Layers of the System

Below is how Rate Limiting can be applied at different layers of the system:

- **Application Layer:**
 - Implementing rate restriction logic inside the application code itself is known as rate limitation at the application layer.
 - It is applicable to all requests that the application processes, irrespective of where they come from or end up.
- **API Gateway Layer:**
 - Setting up rate limiting rules inside the API gateway infrastructure is known as rate limitation at the API gateway layer.
 - It covers incoming requests that the API gateway receives prior to forwarding them to services further down the line.
- **Service Layer:**
 - Rate limiting at the service layer involves implementing rate limiting logic within individual services or microservices.
 - It applies to requests processed by each service independently, allowing for fine-grained control and customization.
- **Database Layer:**
 - Rate limiting at the database layer involves controlling the rate of database queries or transactions.
 - It applies to database operations performed by the application or services, such as read and write operations.

Challenges of Rate Limiting

Below are the challenges of Rate Limiting:

- **Latency:** Rate limitation has the potential to cause latency, particularly when requests are throttled or delayed as a

result of exceeding rate constraints.

- **False Positives:** If the rate limiting logic is flawed or the rate restrictions are very restrictive, rate limiting may unintentionally block valid requests. False positives may cause users to become frustrated and experience service interruptions.
- **Configuration Complexity:** It can be difficult to set up rate limiting rules and thresholds, particularly in systems with a variety of traffic patterns and use cases.
- **Scalability Challenges:** If not appropriately scaled, rate limiting methods themselves may create a bottleneck under excessive load. One of the biggest challenges is making sure rate-limiting systems can manage growing traffic levels without seeing any degradation in performance.